



**UNIVERSIDAD
NACIONAL DE
INGENIERÍA**

Universidad Nacional de Ingeniería

Doctorado en Ciencias e Ingeniería Estadística

Seminarios de Investigación en Estadística, Probabilidad y *Machine Learning* I

LightRAG y los Sistemas RAG

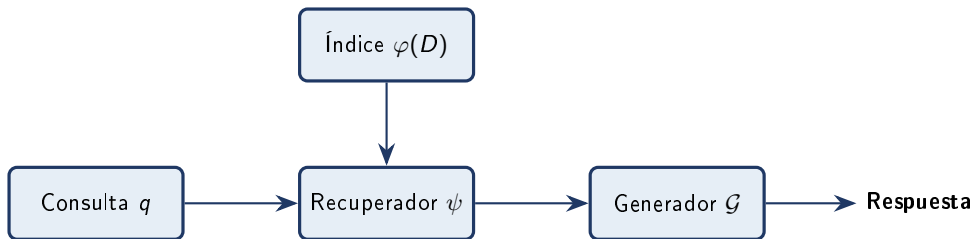
Expositor

Diego Alonso Córdova Ayala

1. ¿Qué es un sistema RAG?
2. Tipos de RAG
3. Comparación de los cinco enfoques
4. LightRAG por dentro
5. Resultados (RQ1 / RQ2 / RQ3)
6. Reproducción con la API
7. Conclusión

1. ¿Qué es un sistema RAG?

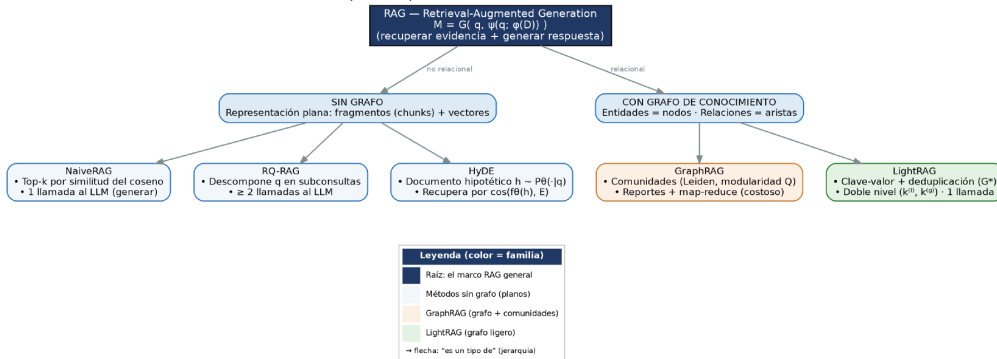
RAG = *Retrieval-Augmented Generation*: **recupera** evidencia y luego **genera** con ella.



$$\mathcal{M}(q; D) = \mathcal{G}(q, \psi(q; \varphi(D)))$$

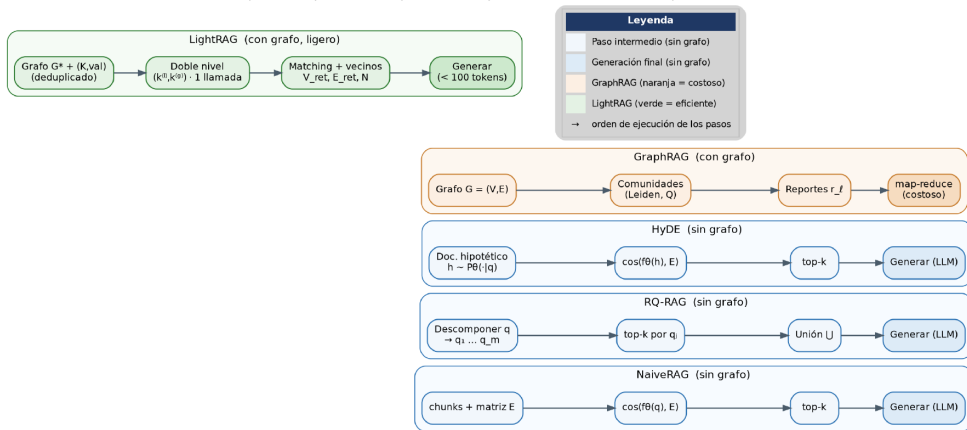
2. Tipos de RAG

Mapa conceptual 1 — Taxonomía de los sistemas RAG



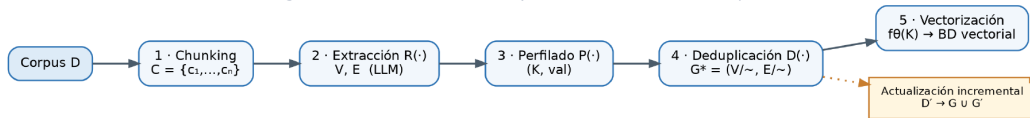
3. Comparación de los cinco enfoques

Mapa conceptual 3 — Pipelines comparados (cada fila = un tipo de RAG)



4. LightRAG por dentro — Indexado φ (una vez)

LightRAG · Fase de INDEXADO φ (una sola vez sobre el corpus)



Una sola pasada sobre el corpus: **chunking** → **extracción** $R(\cdot)$ → **perfilado** $P(\cdot)$ → **deduplicación** $D(\cdot)$
→ **vectorización** f_{θ} .

4. LightRAG por dentro — Recuperación ψ (por consulta)



Una llamada al LLM produce las claves de doble nivel $(k^{(\ell)}, k^{(g)})$; luego *matching* por coseno, vecinos a un salto y generación.

5. RQ1 — Win rate (%) de LightRAG vs. baselines

vs.	Comprehensiveness	Diversity	Empowerment	Overall
NaiveRAG	89	92	91	90
RQ-RAG	92	92	92	92
HyDE	89	89	88	90
GraphRAG	43	43	56	44

Supera ampliamente a los RAG sin grafo ($\approx 90\%$); frente a GraphRAG queda a la par (44% *Overall*) en este *benchmark* reducido ($> 50 = \text{LightRAG mejor}$).

Promedio de 4 datasets (Agriculture, CS, Legal, Mix) · 10 contextos y 11 preguntas por dataset · Juez: LLM en doble orden.

5. RQ2 — ¿Qué sostiene la calidad? La recuperación de doble nivel

Una sola llamada extrae **dos** tipos de claves:

$k^{(\ell)}$ **bajo nivel** — entidades / detalle $\Rightarrow$ *nodos*

$k^{(g)}$ **alto nivel** — temas / concepto $\Rightarrow$ *aristas*

El modo **híbrido** combina ambos y añade el vecindario a un salto; es el que se evalúa en el cuaderno (frente a los modos *local* y *global*).

$$(k^{(\ell)}, k^{(g)}) = \text{LLM}_{kw}(q)$$

$$V_{ret}, E_{ret} = \text{Top}_k \cos(\cdot)$$

$$N = \mathcal{N}(V_{ret}) \cup \mathcal{N}(E_{ret})$$

$$\text{ctx} = \bigoplus_x \text{val}(x)$$

Separar *detalle* y *tema* en una única consulta es lo que da la comprensión de grafo con costo mínimo.

5. RQ3 — Eficiencia de indexado y grafo (medido)

Indicador	GraphRAG	LightRAG
Tiempo de indexado (min)	1,1	1,7
Nodos del grafo	326	490
Aristas del grafo	276	370
Almacenamiento (MB)	en memoria	16,3

LightRAG construye un grafo más denso y lo **persiste en disco** (índice reutilizable e incremental); por consulta usa **una** llamada al LLM frente al *map-reduce* de GraphRAG.

6. Reproducción con la API

▷ [Abrir en Google Colab](#)

⟨/⟩ [Código en GitHub](#)

1. Consigue tu clave en `platform.openai.com` (Billing + API keys).
2. Abre Colab, sube el cuaderno `LightRAG_4datasets_8.ipynb` e instala `lightrag-hku`.
3. Pega tu `OPENAI_API_KEY`, ejecuta *insert* y *query*.

Repositorio oficial: `github.com/HKUDS/LightRAG`

6. Código — Módulos 0–1: instalación y configuración (1/2)

Instala dependencias; carga las claves desde los *Secrets* de Colab; fija los hiperparámetros (10 contextos, 12 preguntas, gpt-4o-mini, TOPK=8) y monta Drive.

```
%pip install -q "lightrag-hku" openai huggingface_hub jsonlines tiktoken pandas nest_asyncio networkx matplotlib
```

```
import os
try:
    from google.colab import userdata
    os.environ["OPENAI_API_KEY"] = userdata.get("OPENAI_API_KEY"); print("[ok] Clave desde Secrets.")
except Exception:
    from getpass import getpass
    if not os.environ.get("OPENAI_API_KEY"):
        os.environ["OPENAI_API_KEY"] = getpass("Pega tu OPENAI_API_KEY: ")

# HF_TOKEN (Hugging Face) acelera la descarga del dataset. Si lo guardaste en Secrets, se carga solo.
try:
    from google.colab import userdata as _ud
    _hf=_ud.get("HF_TOKEN")
    if _hf: os.environ["HF_TOKEN"]=_hf; print("[ok] HF_TOKEN cargado (descargas más rápidas).")
except Exception:
    pass

DATASETS      = ["agriculture","cs","legal","mix"]    # los 4, siempre
MAX_CTX       = 10      # contextos por dataset (balance)
N_PREGUNTAS   = 12      # preguntas por dataset (balance)
MAX_TOKENS_CTX= 3000     # * RECORTE por contexto: indemedado en SEGUNDOS (evita cientos de chunks)
MODELO        = "gpt-4o-mini" # API OpenAI original (no cambia)
EMB_MODEL     = "text-embedding-3-small"
TOPK          = 8
GR_MAX_CHUNKS = 40      # tope de chunks del grafo GraphRAG; bájalo/súbelo aquí
USAR_DRIVE    = True
```

6. Código — Módulos 0–1: instalación y configuración (2/2)

```
BASE="."
if USAR_DRIVE:
    from google.colab import drive; drive.mount("/content/drive")
    BASE="/content/drive/MyDrive/lightrag_ultradomain"
os.makedirs(BASE, exist_ok=True); os.chdir(BASE)
print("Trabajando en:", os.getcwd(), "| datasets:", DATASETS)
```

6. Código — Módulo 2: verificación

Comprueba que los paquetes importan y que la API responde (un *ping* de chat y la dimensión de los *embeddings*) antes de gastar cuota.

```
import importlib
for pkg in ["lightrag", "openai", "huggingface_hub", "tiktoken", "networkx", "nest_asyncio", "matplotlib"]:
    try: importlib.import_module(pkg); print(f"import {pkg}: OK")
    except Exception as e: print(f"import {pkg}: ! {e}")
from openai import OpenAI as _OA
_c=_OA()
try:
    _c.chat.completions.create(model=MODEL0, messages=[{"role": "user", "content": "ping"}], max_tokens=1)
    print("chat API: OK, emb dim:", len(_c.embeddings.create(model=EMB_MODEL, input=["ping"]).data[0].
        embedding))
    print("\n[ok] Listo.")
except Exception as e: print("! API FALLA ->", e)
```

6. Código — Módulo 3 (utilidades) · I: base numérica (1/2)

Funciones base: `chat()` es una llamada de completión; `embed()` embebe por lotes y normaliza (así el producto punto es el coseno); `retrieve()` recupera los top-k por coseno.

```
import time, random, re, json, glob
import numpy as np
from openai import OpenAI
from tiktoken import get_encoding
from concurrent.futures import ThreadPoolExecutor

client = OpenAI(max_retries=5)
enc = get_encoding("cl100k_base")
S = {}          # estado del dataset actual (uniq, PREG, CH, EMB, CLS)
STATS = {}      # métricas por dataset (para la tabla de eficiencia)

def _retry(fn,*a,**k):
    for t in range(6):
        try: return fn(*a,**k)
        except Exception:
            if t==5: raise
            time.sleep(min(2**t+random.random(),30))

def chat(prompt, temp=0.0, system=None):
    msgs=([{"role":"system","content":system}] if system else [])+[{"role":"user","content":prompt}]
    r=_retry(client.chat.completions.create, model=MODELO, messages=msgs, temperature=temp)
    return r.choices[0].message.content

def embed(texts):
    texts=[t if (t and t.strip()) else " " for t in texts]
    vecs=[]
    for j in range(0,len(texts),64):
        r=_retry(client.embeddings.create, model=EMB_MODEL, input=texts[j:j+64])
        vecs += [d.embedding for d in r.data]
    a=np.array(vecs,dtype=np.float32); return a/(np.linalg.norm(a,axis=1,keepdims=True)+1e-9)
```

6. Código — Módulo 3 (utilidades) · I: base numérica (2/2)

```
def split_chunks(text, size=1200, overlap=100):
    t=enc.encode(text); out=[]; i=0
    while i<len(t):
        out.append(enc.decode(t[i:i+size])); i+=size-overlap
    return out

def retrieve(query, k=TOPK):
    v=embed([query])[0]; sims=S["EMB"]@v
    idx=np.argsort(sims)[::-1][:k]; return [S["CH"][i] for i in idx]

def gen_answer(q, ctxs):
    return chat("Answer the question using ONLY the context.\n\nContext:\n"+" \n\n".join(ctxs)+"\n\nQuestion: "+q, 0.0)
```

6. Código — Módulo 3 (utilidades) · II: paralelo y LLM-juez (1/2)

`responder_todos()` corre un RAG sobre todas las preguntas en paralelo; el LLM-juez (`_jp,_w,win_rate`) evalúa en ambos órdenes y cuenta las victorias de LightRAG.

```
def responder_todos(fn, nombre, workers=8):
    PREG=S["PREG"]; CLS=S["CLS"]
    def _one(q):
        try: return {"query":q,"result":fn(q)}
        except Exception as e: return {"query":q,"result":f"[error: {e}]"}
    if workers>1:
        with ThreadPoolExecutor(max_workers=workers) as ex: out=list(ex.map(_one,PREG))
    else: out=[_one(q) for q in PREG]
    json.dump(out, open(f"{CLS}_{nombre}.json","w"), ensure_ascii=False)
    return out

def _mb(d): return sum(os.path.getsize(p) for p in glob.glob(os.path.join(d,"**"),recursive=True) if os.path.isfile(p))/1e6

# ---- LLM-juez (win rate en ambos órdenes) ----
DIMS=["Comprehensiveness","Diversity","Empowerment","Overall"]
SYSJ="You are an expert evaluating two answers based on Comprehensiveness, Diversity, Empowerment."
def _jp(q,a1,a2):
    return ("Evaluate two answers by: Comprehensiveness, Diversity, Empowerment; then an Overall winner.\n"
            "Question:\n"+q+"\n\nAnswer 1:\n"+str(a1)+"\n\nAnswer 2:\n"+str(a2)+"\n\n"
            'Output ONLY JSON: {"Comprehensiveness":{"Winner":"Answer 1|Answer 2"},"'
            '"Diversity":{"Winner":"..."},"Empowerment":{"Winner":"..."},"Overall":{"Winner":"..."}}')
def _w(q,a1,a2):
    try:
        d=json.loads(re.search(r"\{.*\}", chat(_jp(q,a1,a2),0.0,SYSJ), re.S).group(0))
        return {k:("1" if "1" in str(d.get(k,{ })).get("Winner","")) else "2" for k in DIMS}
    except Exception:
```

6. Código — Módulo 3 (utilidades) · II: paralelo y LLM-juez (2/2)

```
        return {k:"2" for k in DIMS}
def win_rate(fileLR, fileBase, workers=8):
    dLR=json.load(open(fileLR)); dB=json.load(open(fileBase))
    Q=[x["query"] for x in dLR]; L=[x["result"] for x in dLR]; B=[x["result"] for x in dB]
    def _pair(t):
        q,a,b=t; return _w(q,a,b), _w(q,b,a)
    with ThreadPoolExecutor(max_workers=workers) as ex:
        pares=list(ex.map(_pair, zip(Q,L,B)))
    wins={d:0 for d in DIMS}; tot=0
    for r1,r2 in pares:
        for d in DIMS: wins[d]+=(r1[d]=="1")+(r2[d]=="2")
        tot+=2
    return {d:round(100*wins[d]/tot,1) for d in DIMS}
```

6. Código — Módulo 3 (utilidades) · III: preparación del dataset

`preparar()` descarga UltraDomain, deduplica y recorta los contextos, genera las preguntas con el LLM, trocea el corpus y lo embebe.

```
def preparar(CLS):
    from huggingface_hub import hf_hub_download
    import jsonlines
    ruta=hf_hub_download(repo_id="TommyChien/UltraDomain", filename=f"{CLS}.jsonl", repo_type="dataset")
    seen=set(); uniq=[]
    with jsonlines.open(ruta) as r:
        for o in r:
            ctx=o.get("context")
            if ctx and ctx not in seen: seen.add(ctx); uniq.append(ctx)
    if MAX_CTX: uniq=uniq[:MAX_CTX]
    # * Recorta cada contexto a MAX_TOKENS_CTX tokens: así 1 documento gigante NO genera cientos de chunks.
    uniq=[enc.decode(enc.encode(c)[:MAX_TOKENS_CTX]) for c in uniq]
    ids=enc.encode("\n".join(uniq))
    desc=enc.decode(ids[1000:1500])+"\n"+enc.decode(ids[max(0,len(ids)-1500):max(0,len(ids)-1000)])
    n_pares=max(1,N_PREGUNTAS//5)
    pq=("Given the following description of a dataset:\n\n"+desc+"\n\n"
        f"Identify {n_pares} (user, task) combinations. For each, generate 5 questions that require a "
        "high-level understanding of the entire dataset.\n"
        "Output ONLY the questions, one per line, as: - Question N: <question>")
    txt=chat(pq,1.0)
    PREG=re.findall(r"- Question \d+:\s*(.+)", txt)[:N_PREGUNTAS]
    if len(PREG)<N_PREGUNTAS:
        PREG=(PREG+[l.strip("- ").strip() for l in txt.splitlines() if "?" in l])[:N_PREGUNTAS]
    CH=[]
    for ctx in uniq: CH+=split_chunks(ctx)
    CH=[x for x in CH if x and x.strip()]
    EMB=embed(CH)
    S.clear(); S.update(CLS=CLS, uniq=uniq, PREG=PREG, CH=CH, EMB=EMB)
    STATS.setdefault(CLS, {}).update(ctx=len(uniq), preg=len(PREG))
    print(f"  preparado {CLS}: {len(uniq)} contextos, {len(PREG)} preguntas, {len(CH)} chunks")
```

6. Código — §A–§C: NaiveRAG, RQ-RAG y HyDE

Los tres sistemas sin grafo: NaiveRAG (top-k directo), RQ-RAG (descompone en subpreguntas y une) y HyDE (documento hipotético y recupera por su *embedding*).

```
def run_naive():
    def naive_rag(q):
        return gen_answer(q, retrieve(q, k=TOPK))
    responder_todos(naive_rag, "naive")    # -> {CLS}_naive.json

def run_rqrag():
    def rq_rag(q):
        raw=chat("Decompose the question into up to 3 focused sub-questions. "
                 "Return ONLY a JSON list of strings.\nQuestion: "+q, 0.3)
        try: subs=json.loads(re.search(r"\[.*\]", raw, re.S).group(0))
        except Exception: subs=[q]
        ctxs=[]
        for s in subs[:3]: ctxs += retrieve(s, k=5)
        return gen_answer(q, list(dict.fromkeys(ctxs)))
    responder_todos(rq_rag, "rqrag")    # -> {CLS}_rqrag.json

def run_hyde():
    def hyde(q):
        h=chat("Write a concise passage that would answer the question.\nQuestion: "+q, 1.0)
        return gen_answer(q, retrieve(h, k=TOPK))
    responder_todos(hyde, "hyde")    # -> {CLS}_hyde.json
```

6. Código — §D: GraphRAG (1/2)

Extrae entidades y relaciones por LLM, construye el grafo (networkx), detecta comunidades (Louvain), resume cada una y responde por *map-reduce*.

```
def run_graphrag():
    import networkx as nx
    from collections import defaultdict
    CLS=S["CLS"]; CH=S["CH"]
    def _extraer(ch):
        p=('Extract the key entities and their relationships from the text. Return ONLY JSON: '
          '{"entities":[{"name":"..","desc":".."}],"relations":[{"source":"..","target":".."}]}\n\nText:\n'+ch)
        try:
            d=json.loads(re.search(r"\{.*\}", chat(p,0.0), re.S).group(0)); return d.get("entities",[]),d.get("relations",[])
        except Exception: return [],[]
    t0=time.time(); G=nx.Graph(); ent_desc=defaultdict(list)
    with ThreadPoolExecutor(max_workers=8) as ex:
        for ents,rels in ex.map(_extraer, CH[:GR_MAX_CHUNKS]):
            for e in ents:
                n=str(e.get("name","")).strip()
                if n: ent_desc[n].append(str(e.get("desc",""))); G.add_node(n)
            for r in rels:
                s=str(r.get("source","")).strip(); t=str(r.get("target","")).strip()
                if s and t: G.add_edge(s,t)
    coms=list(nx.community.louvain_communities(G, seed=42)) if G.number_of_nodes() else []
    def _reporte(com):
        lin=[]
        for n in list(com)[:40]: lin.append("- "+n+": "+" ".join(ent_desc.get(n,[])[:2]))
        return chat("Write a concise report describing this community of related entities and its main themes.\n\n"+" ".join(lin),0.3)
    with ThreadPoolExecutor(max_workers=8) as ex: REP=list(ex.map(_reporte, coms)) if coms else []
    S["REP"]=REP; S["REPE"]=embed(REP) if REP else None
```

6. Código — §D: GraphRAG (2/2)

```
STATS[CLS].update(idx_gr=(time.time()-t0)/60, gr_nodes=G.number_of_nodes(), gr_edges=G.number_of_edges())
def graphrag_ans(q):
    REP=S["REP"]; REPE=S["REPE"]
    if not REP: return "[GraphRAG sin comunidades]"
    v=embed([q])[0]; idx=np.argsort(REPE@v)[::-1][:4]; sel=[REP[i] for i in idx]
    par=[]
    for rep in sel:
        a=chat("Using ONLY this community report, give a partial answer; if not relevant reply exactly N/A.\n\nReport:\n"+rep+"\n\nQuestion: "+q,0.0)
        if a and a.strip()[:3]!="N/A": par.append(a)
    if not par: par=sel[:3]
    return chat("Combine these partial answers into ONE comprehensive answer.\n\n"+"".join(par)+"\n\nQuestion: "+q,0.0)
responder_todos(graphrag_ans, "graphrag", workers=4) # -> {CLS}_graphrag.json
```

6. Código — §E: LightRAG (el sistema propuesto)

Indexa con la librería `lightrag-hku` (fase φ) y consulta en los modos `hybrid/local/global`; mide tiempo, nodos y aristas del grafo.

```
async def run_lightrag():
    from lightrag import LightRAG, QueryParam
    from lightrag.llm.openai import gpt_4o_mini_complete, openai_embed
    from lightrag.kg.shared_storage import initialize_pipeline_status
    from lightrag.utils import setup_logger
    import networkx as nx
    setup_logger("lightrag", level="WARNING")
    CLS=S["CLS"]; LR_DIR=os.path.join(BASE, f"lightrag_{CLS}"); os.makedirs(LR_DIR, exist_ok=True)
    rag=LightRAG(working_dir=LR_DIR, embedding_func=openai_embed,
                 llm_model_func=gpt_4o_mini_complete, chunk_token_size=1200)
    await rag.initialize_storages(); await initialize_pipeline_status()
    t0=time.time(); await rag.ainsert(S["uniq"]); T_LR=time.time()-t0
    import asyncio as _aio
    for mode in ["hybrid","local","global"]: # sin "mix" (no se usa en las tablas)
        tareas=[rag.aquery(q, param=QueryParam(mode=mode)) for q in S["PREG"]]
        res=await _aio.gather(*tareas, return_exceptions=True) # todas las preguntas EN PARALELO
        out=[{"query":q,"result":(str(r) if not isinstance(r,Exception) else f"[error: {r}]")}
              for q,r in zip(S["PREG"],res)]
        json.dump(out, open(f"{CLS}_lr_{mode}.json","w"), ensure_ascii=False)
    try:
        GL=nx.read_graphml(os.path.join(LR_DIR,"graph_chunk_entity_relation.graphml")); LR_N,LR_E=GL.number_of_nodes(),GL.number_of_edges()
    except Exception: LR_N,LR_E=0,0
    STATS[CLS].update(idx_lr=T_LR/60, mb_lr=round(_mb(LR_DIR),1), lr_nodes=LR_N, lr_edges=LR_E)
    try:
        await rag.finalize_storages() # libera el estado global de LightRAG entre datasets
    except Exception:
        pass
```

6. Código — §F y Módulo 4: evaluación y runner (1/3)

`evaluar()` calcula el win rate de LightRAG-hybrid frente a cada baseline; el *runner* recorre los 4 datasets en un solo *event loop* y reanuda saltando los ya hechos.

```
def evaluar(CLS):
    import pandas as pd
    baselines={"NaiveRAG":f"{CLS}_naive.json","RQ-RAG":f"{CLS}_rqrag.json",
               "HyDE":f"{CLS}_hyde.json","GraphRAG":f"{CLS}_graphrag.json"}
    LR=f"{CLS}_lr_hybrid.json"
    rq1={n: win_rate(LR,p) for n,p in baselines.items() if os.path.exists(p)}
    df=pd.DataFrame(rq1).T[DIMS]
    df.to_csv(f"RQ1_{CLS}.csv") # guarda el resultado (para no repetir mañana)
    json.dump(STATS.get(CLS,{}), open(f"STATS_{CLS}.json","w"))
    return df

import asyncio, nest_asyncio; nest_asyncio.apply()
import pandas as pd
RES={}

async def _run_all():
    for CLS in DATASETS:
        print(f"\n##### {CLS} #####")
        try:
            if os.path.exists(f"RQ1_{CLS}.csv"): # ya hecho antes -> 0 llamadas API
                try: STATS[CLS]=json.load(open(f"STATS_{CLS}.json"))
                except Exception: pass
                RES[CLS]=pd.read_csv(f"RQ1_{CLS}.csv", index_col=0)
                print(f" {CLS} ya estaba hecho: se reutiliza (0 API)"); continue
            preparar(CLS)
            run_naive(); print(" §A NaiveRAG ok")
```

6. Código — §F y Módulo 4: evaluación y runner (2/3)

```
run_rqrag();    print(" §B RQ-RAG ok")
run_hyde();    print(" §C HyDE ok")
run_graphrag(); print(" §D GraphRAG ok")
await run_lightrag(); print(" §E LightRAG ok")  # AWAIT: mismo loop para los 4
RES[CLS]=evaluar(CLS); print(f" [ok] {CLS} completado")
except Exception as e:
    print(f" ! Falló {CLS}: {e}")
```

6. Código — §F y Módulo 4: evaluación y runner (3/3)

```
asyncio.run(_run_all())  
print("\nDatasets completados:", list(RES))
```

6. Código — Módulo 5: tabla comparativa (PDF) (1/2)

Lee los CSV, promedia por baseline (*groupby-mean*) y arma el PDF único con los paneles de calidad (win rate) y de eficiencia.

```
import pandas as pd, numpy as np
import matplotlib.pyplot as plt
from matplotlib.backends.backend_pdf import PdfPages

import json as _json
fr=[]; metas=[]
for cls in DATASETS:
    if os.path.exists(f"RQ1_{cls}.csv"):
        # lee de DISCO: combina aunque hayas corrido en varios días
        d=pd.read_csv(f"RQ1_{cls}.csv", index_col=0)
        d=d.reset_index().rename(columns={d.reset_index().columns[0]:"baseline"}); d["ds"]=cls; fr.append(d)
        if os.path.exists(f"STATS_{cls}.json"):
            try: metas.append(_json.load(open(f"STATS_{cls}.json")))
            except Exception: pass
if not fr:
    raise RuntimeError("No hay resultados en disco todavía. Corre §4 primero.")
rq1=pd.concat(fr).groupby("baseline")[DIMS].mean().round(0)
orden=[b for b in ["NaiveRAG", "RQ-RAG", "HyDE", "GraphRAG"] if b in rq1.index]; rq1=rq1.loc[orden]
def _avg(k):
    vals=[m[k] for m in metas if k in m]; return float(np.mean(vals)) if vals else 0.0
B_rows=["Tiempo de indexado (min)", "Nodos del grafo", "Aristas del grafo", "Almacenamiento (MB)"]
B_gr=[f"_{avg('idx_gr'):.1f}", f"_{avg('gr_nodes'):.0f}", f"_{avg('gr_edges'):.0f}", "en memoria"]
B_lr=[f"_{avg('idx_lr'):.1f}", f"_{avg('lr_nodes'):.0f}", f"_{avg('lr_edges'):.0f}", f"_{avg('mb_lr'):.1f}"]

AZUL="#2E75B6"; AZULC="#DEEAF6"; VERDE="#1E7B34"; nds=len(fr)
def estilo(ax, tabla, resalta_col=None):
    tabla.auto_set_font_size(False); tabla.set_fontsize(11); tabla.scale(1,1.6)
    for (row,col),cell in tabla.get_celld().items():
        cell.set_edgecolor("white")
        if row==0: cell.set_facecolor(AZUL); cell.set_text_props(color="white", fontweight="bold")
        else: cell.set_facecolor(AZULC if row%2 else "white")
        if resalta_col is not None and col==resalta_col and row>0: cell.set_text_props(color=VERDE, fontweight="bold")
```

6. Código — Módulo 5: tabla comparativa (PDF) (2/2)

```
fig=plt.figure(figsize=(11,8.5)); fig.patch.set_facecolor("white")
fig.suptitle("Comparativa de sistemas RAG UltraDomain (promedio de %d datasets)"%nds, fontsize=15, fontweight="bold", y=0.97)

axA=fig.add_axes([0.06,0.55,0.88,0.32]); axA.axis("off")
axA.set_title("Calidad win rate (%) de LightRAG frente a cada baseline (>50 = LightRAG mejor)", fontsize=12, fontweight="bold", loc="left", pad=10)
colsA=["Baseline"]+DIMS
datA=[[b]+[f"{rq1.loc[b,c]:.0f}" for c in DIMS] for b in rq1.index]
tA=axA.table(cellText=datA, colLabels=colsA, loc="center", cellLoc="center"); estilo(axA,tA, resalta_col=len(colsA)-1)

axB=fig.add_axes([0.18,0.16,0.64,0.28]); axB.axis("off")
axB.set_title("Eficiencia indicadores medidos (GraphRAG vs. LightRAG)", fontsize=12, fontweight="bold", loc="left", pad=10)
datB=[[B_rows[i],B_gr[i],B_lr[i]] for i in range(len(B_rows))]
tB=axB.table(cellText=datB, colLabels=["Indicador","GraphRAG","LightRAG"], loc="center", cellLoc="center"); estilo(axB,tB, resalta_col=None)

fig.text(0.06,0.07,"Benchmark UltraDomain (%d contextos y %d preguntas por dataset). Juez: LLM en doble orden."%(int(_avg('ctx')),int(_avg('preg'))),
, fontsize=9, color="#555555")

PDF="Comparativa_RAG_LightRAG.pdf"; PNG="Comparativa_RAG_LightRAG.png"
with PdfPages(PDF) as pp: pp.savefig(fig, bbox_inches="tight")
fig.savefig(PNG, dpi=160, bbox_inches="tight")
plt.show()
print("\n[ok] ÚNICO archivo para el Beamer:", os.path.abspath(PDF))
```

Conclusión

LightRAG une **grafo de conocimiento** y **recuperación de doble nivel**: casi la calidad del enfoque de grafo, con una sola llamada por consulta y actualización incremental.

Guo et al., EMNLP 2025 · github.com/HKUDS/LightRAG